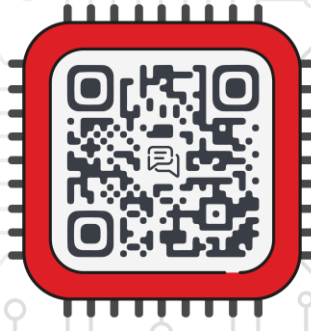




8



10



1200 sp

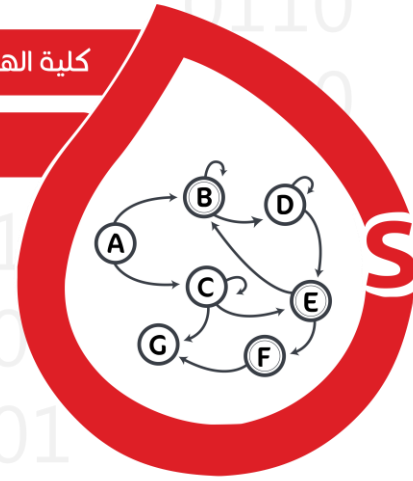
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Context Free Grammar Part 2

د. هبة حاتم



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 01/06/2025

اللغات الصورية

أشجار التحليل / الاشتقاق Parse Tree (derivation tree)

أشجار التحليل أو أشجار الاشتقاق لها جذر root و تكون مرتبة و هي تمثيل بصري (رسومي) لاشتقاق strings مختلفة عن طريق ال CFG، أي تظهر الاشتقاق خطوة بخطوة.

كل CFG يمكن التعبير عنها بشجرة اشتقاق

■ الجذر هو الحالة الابتدائية start symbol

■ كل عقدة داخلية عبارة عن متغير non-terminal و تسمى Vertex

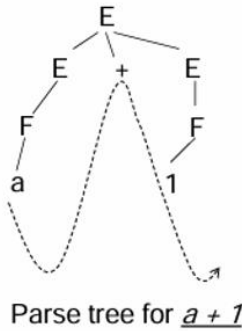
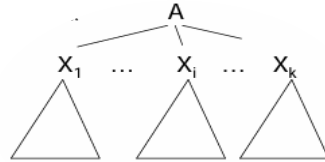
■ كل عقدة ورقة هي terminal

(لذلك تم تسميتهم terminals لأنهم نهاية الشجرة أما ال non-terminals كلما عوضنا بال PR الخاصة بهم حصلنا على سلسلة جديدة)

■ من أجل $A \Rightarrow X_1..X_i..X_k : PR$

ترميز الأبناء من اليسار إلى اليمين

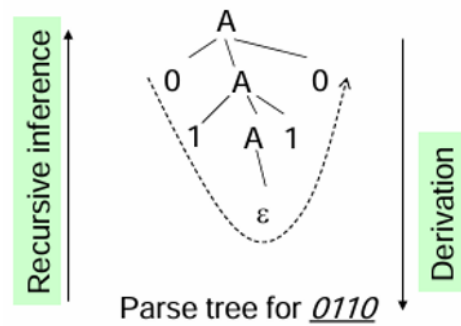
نحدد أبناء عقدة معينة بحسب ال PR



G:

$$E \rightarrow E + E \mid E * E \mid (E) \mid F$$

$$F \rightarrow aF \mid bF \mid 0F \mid 0 \mid 1 \mid a \mid b$$



G:

$$A \rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \epsilon$$

أمثلة:

شجرة الاشتقاق الأيسر وشجرة الاشتقاق الأيمن

Left Derivation Tree and Right Derivation Tree

كما ذكرنا في المحاضرة السابقة شجرة الاشتقاق الأيسر يتم الحصول عليها بتطبيق ال PR للمتغير في أقصى اليسار كل مرة أما الأخرى فنبدأ من اليمين.

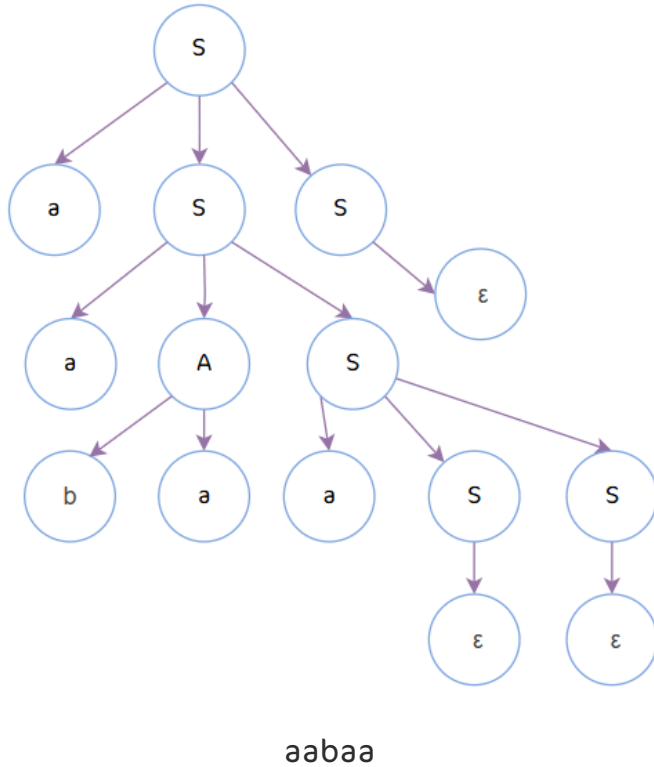
- التمييز بين الاشتقاق من اليمين و الاشتقاق من اليسار أمر مهم، لأن معظم المحلات يتم تحديد طريقة تحويل الدخل عن طريق كود برمجي موافق لكل قاعدة حيث يتم تنفيذ هذا الكود في كل مرة يتم تطبيق القاعدة فيها و بالتالي فإن التمييز بينهم مهم لتحديد ترتيب و أولوية تنفيذ الأكواد البرمجية (مثل تنفيذ عملية الضرب قبل الجمع)
- لكل شجرة اشتقاق يسرى هناك شجرة اشتقاق يمنى أي لكل CFG هناك شجرتا اشتقاق.

مثال:

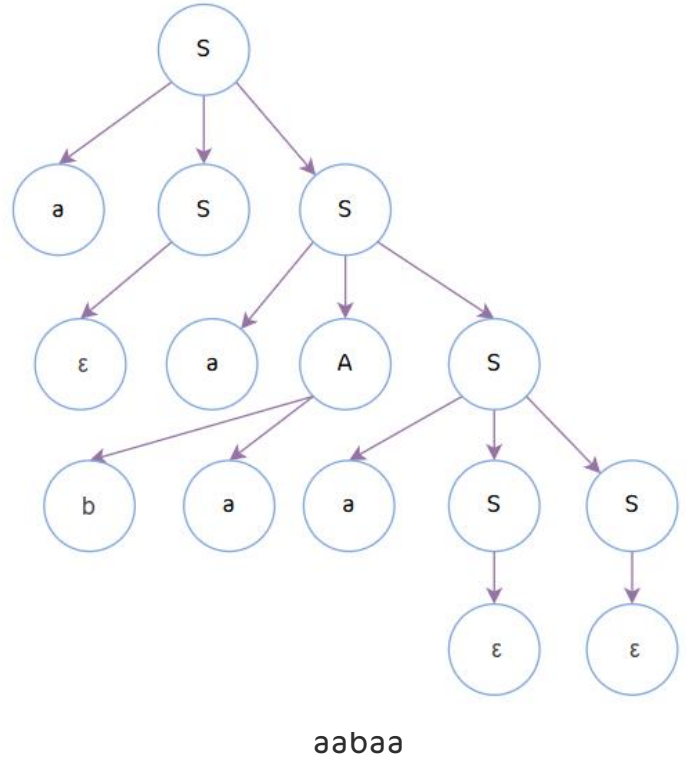
لاشتقاق السلسلة aabaa من القاعدة

$$S \rightarrow aAS \mid aSS \mid \varepsilon, A \rightarrow SbA \mid ba$$

شجرة الاشتقاق من اليسار



شجرة الاشتقاق من اليمين



الغموض في CFGs

تكون ال CFGs غامضة إذا كان هناك على الأقل سلسلة واحدة مشتقة من هذه القاعدة لها أكثر من شجرة اشتقاق يسرى مختلفة distinct أي تختلف بالبنية او الهرمية.

ملاحظة:

يمكن القول أيضا انها غامضة إذا وجدت سلسلة مولدة من هذه القاعدة لها أكثر من شجرة اشتقاق معنى و ذلك لأنه كل شجرة اشتقاق يسرى لها شجرة اشتقاق معنى مطابقة لها

مثال على ذلك:

$$S \rightarrow AS \mid \varepsilon$$

$$A \rightarrow A1 \mid 0A1 \mid 01$$

$S \rightarrow AS$
 $\rightarrow 0A1S$
 $\rightarrow 0A11S$
 $\rightarrow 00111S$
 $\rightarrow 00111$

$S \rightarrow AS$
 $\rightarrow A1S$
 $\rightarrow 0A11S$
 $\rightarrow 00111S$
 $\rightarrow 00111$

نلاحظ أن السلسلة 00111 أمكن اشتقاقها بطريقتين مختلفتين.

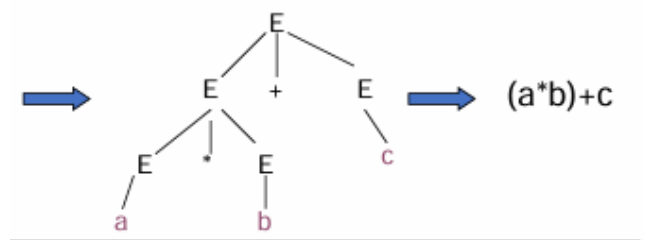
لماذا يهمنا معرفة وإزالة الغموض why does ambiguity matter

سنوضح ذلك بمثال:

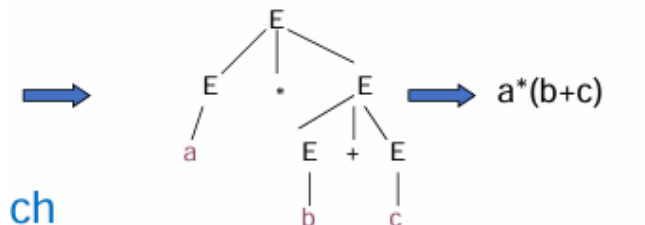
لدينا ال PR التالية $E \rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c \mid 0 \mid 1$

و نريد اشتقاق السلسلة $a*b+c$ منها (سنطبق الاشتقاق من اليسار)

• $E \rightarrow E + E \rightarrow E * E + E \rightarrow a * b + c$



• $E \rightarrow E * E \rightarrow a * E \rightarrow a * E + E \rightarrow a * b + c$



نلاحظ أنه نتج لدينا شجرتا اشتقاق مختلفتين، وستختلف القيمة الفعلية النهائية بحسب أي من الأشجار سيتم اعتماده (إذا أسقطنا الأمر على الكود البرمجي) حيث يتم تفسير الشجرة الأولى بـ $(a*b)+c$

أما الشجرة الثانية ستفسر بـ $a*(b+c)$

(نبدأ بتفسير interpretation - تحويل الشجرة لنص - من المستوى الأخير صعوداً لمستوى العقدة)

مثال آخر قريب له:

باعتبار القاعدة $G = (\{S\}, \{a, b, +, *\}, P, S)$ $G = (\{S\}, \{a, b, +, *\}, P, S)$

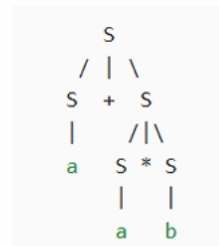
حيث P contains: $S \rightarrow S + S \mid S * S \mid a \mid b$

السلسلة $a+a*b$ يمكن توليدها بطريقتين مختلفتين، مما يُظهر الغموض.

Derivation 1

1. $S \Rightarrow S+S$
2. $\Rightarrow a+S$
3. $\Rightarrow a+S*S$
4. $\Rightarrow a+a*S$
5. $\Rightarrow a+a*b$

• Parse Tree structure

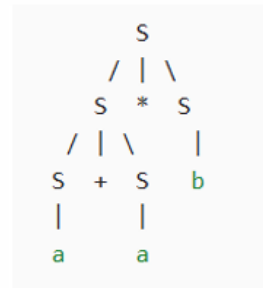


تفسر بـ $a+(a*b)$ ، يتم تنفيذ الضرب قبل الجمع

Derivation 2

1. $S \Rightarrow S*S$
2. $\Rightarrow S+S*S$
3. $\Rightarrow a+S*S$
4. $\Rightarrow a+a*S$
5. $\Rightarrow a+a*b$

Parse Tree structure

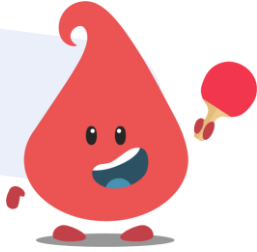


تفسر بـ $(a+a)*b$ ، يُنفذ الجمع قبل الضرب.

إزالة الغموض Removing Ambiguity

إزالة الغموض من القاعدة يعني التعديل على ال PR الخاصة فيها للحرص على أن كل سلسلة تولدها هذه القاعدة لها شجرة اشتقاق واحدة مميزة unique (أو بتعبير مكافئ: لها زوج اشتقاق أيمن / أيسر مميز) وهذا يزيل الاختلافات بترجمة السلسلة الواحدة وهو أمر مهم كما ذكرنا سابقاً للغات البرمجة حيث الغموض فيها يؤدي لسلوك غير متوقع، ومن الأساس فإن ال compilers لا تدعم القواعد الغامضة.

أهداف إزالة الغموض



- ضمان التحليل الحتمي (تفسير واحد فقط لكل سلسلة)
 - فرض أولوية العوامل operator precedence (مثل الضرب قبل الجمع)
 - حل تعارضات الاتجاهية Resolve associativity conflicts
- مثال:
- الضرب هو left-associative , $a*b$ نضرب a ب b وكذلك الجمع و الطرح
- الرفع لقوة right-associative (a^b)

Associativity and precedence

أسبقية العوامل Precedence:

- تقسيم القواعد إلى مستويات هرمية, مثل:

Expression $\rightarrow$ Term + Term , Term $\rightarrow$ Factor * Factor , Factor $\rightarrow$ a|b|(Expression)

حيث تكون العملية التي يتم تعويضها في النهاية هي ذات الأسبقية (الأولوية) الأعلى, مثال:

1. عملية الضرب لها الأولوية الأعلى $E \rightarrow E+T$, $T \rightarrow T*T$

2. عملية الجمع لها الأولوية الأعلى $E \rightarrow E*T$, $T \rightarrow T+T$

الاتجاهية Associativity:

- استخدام قواعد ذات استدعاء أيسر (left-recursive) أو أيمن (right-recursive) لفرض ترتيب التنفيذ.

- تتحدد الاتجاهية للعمليات حسب ال production rules المعطاة, مثال:

$E \rightarrow E+T$ (left-recursive)

$E \rightarrow T+E$ (right-recursive)

مثال:

$E \rightarrow E - T \mid E * T$, $T \rightarrow U + T$, $U \rightarrow a$

- العمليتين (- و *) left-recursive و الجمع + right-recursive

- العمليتين (- و *) لهما نفس الأولوية أما الجمع له الأولوية الأعلى.

ملاحظة

إذا أتى سؤال عن أولويات العمليات أو يتضمنها دون ذكر قواعد أو ترتيب محدد لها نعتمد الأولويات الأساسية المعروفة، و هي بالترتيب من الأعلى أولوية للأقل :

$\wedge \leftarrow * / \div \leftarrow + -$

طرق لإزالة الغموض Methods to Remove Ambiguity

1. إعادة هيكلة قواعد النحو Restructure Grammar Rules

■ أسبقية العوامل (Operator Precedence)

تقسيم القواعد إلى مستويات هرمية، مثل:

 $\text{Expression} \rightarrow \text{Term} + \text{Term}, \text{Term} \rightarrow \text{Factor} * \text{Factor}, \text{Factor} \rightarrow a \mid b \mid (\text{Expression})$

■ الاتجاهية (Associativity)

استخدام قواعد ذات استدعاء أيسر (left-recursive) أو أيمن (right-recursive) لفرض ترتيب التنفيذ

مثال: الجمع left-associative ($E \rightarrow E + T \mid T$)

2. إضافة الأقواس add parentheses

تعريف الأقواس بشكل صريح لتجاوز أسبقية العوامل الافتراضية

3. تعريف متغيرات non-terminals جديدة

تجزئة القواعد الغامضة إلى قواعد فرعية غير غامضة

إزالة الغموض في تقييم التعابير Removing Ambiguity in Expression Evaluations

مثال:القاعدة الغامضة $E \rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c \mid 0 \mid 1$

تعديلها (قاعدة غير غامضة):

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow I \mid (E) \\ I &\rightarrow a \mid b \mid c \mid 0 \mid 1 \end{aligned}$$

شرح:

هدفنا الفصل بين ال terminals و ال non-terminals في ال PR، و تحديد الأولويات بين العمليات حيث العملية ذات الأولوية الأكبر سيتم استبدالها في النهاية، فمثلا سيتم استبدال الجمع أولا ثم الضرب ثم الأقواس، وهذا موافق تماما لبنية الشجرة حيث ذكرنا أنه يتم تفسيرها بدءاً من المستوى الأخير الآن نبدأ بالحل:

نلاحظ ان العملية الأقل أهمية من الموجودين هي الجمع لذلك سنقوم بالفصل كالتالي: (تذكر ان الجمع left-associative لذلك سنضع المتغير الجديد لليمين)

$$E \Rightarrow E + T \mid T$$

نقوم بتعريف T حيث سنضع فيها العملية التالية الأكثر أولوية مباشرة من الجمع وهي الضرب في هذا المثال:

$$T \Rightarrow T * F \mid F$$

نقوم بتعريف F:

$$F \Rightarrow I \mid (E)$$

و نهاية نعرف | و هي ال PR التي تعطي ال Terminals و هي النهاية.

$I \Rightarrow a | b | c | 0 | 1$

How will this avoid Ambiguity

- فصلنا ال terminals عن ال non-terminal
- حددنا أولويات العمليات
- حددنا ال associativity

- مع ذلك لا يمكن إزالة الغموض لبعض لغات ال CFL
 - يقال عن ال CFL أنها غامضة بطبيعتها inherently Ambiguous إذا كانت كل قاعدة CFG تولد هذه اللغة غامضة

مثال: $L = \{ a^n b^n c^m d^m \mid n, m \geq 1 \} \cup \{ a^n b^m c^m d^n \mid n, m \geq 1 \}$

طرق لتبسيط القاعد الحرة من السياق

Simplification of Context-Free Grammar

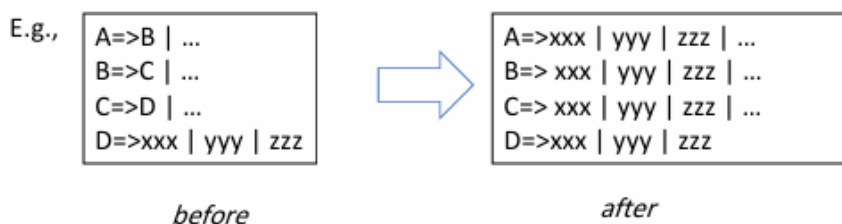
1. إزالة القواعد الأحادية Removal of Unit Productions

تعريف ال Unit production هو كل PR من الشكل $A \rightarrow B$ حيث A و B هم non-terminals

إجرائية إزالة القواعد الأحادية

1. تحديد القواعد الأحادية
2. استبدال الاشتقاقات غير المباشرة حيث لكل قاعدة أحادية $A \rightarrow B$:
 - نضيف قاعدة جديدة $A \rightarrow x$ لكل قاعدة $B \rightarrow x$
 حيث x ممكن ان يكون terminal أو non-terminal أو ϵ (وهو terminal أيضاً)
3. نحذف القاعدة الأحادية
3. نعيد الخطوات حتى لا يبقى أي Unit productions

مثال 1:



مثال 2:

لدينا ال PRs التالية :

$E \rightarrow T \mid E+T$

$T \rightarrow F \mid T * F$

$$F \rightarrow I \mid (E)$$

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1$$

الحل

نبدأ بالحالة الابتدائية و نحدد القاعدة الأحادية $E \rightarrow T$

نعوض الطرف الأيمن بما يوافقه (ال PR الخاصة به): $E \rightarrow F \mid T^*F$

و الآن أصبحت القاعدة الأحادية الموجودة هي: $E \rightarrow F \mid T^*F \mid E+T$

نعوض $E \rightarrow I \mid (E) \mid T^*F \mid E+T : F$

ثم $E \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1 \mid (E) \mid T^*F \mid E+T : I$

لم يبق أي قاعدة أحادية بالتالي وصلنا للشكل المبسط المطلوب ل E

بنفس الطريقة نقوم بتبسيط باقي ال PRs.



2. خوارزمية الزوج الأحادي لإزالة القاعدة الأحادية: The Unit Pair Algorithm: to remove unit productions

فرضاً لدينا $A \rightarrow B1 \rightarrow B2 \rightarrow \dots \rightarrow Bn \rightarrow \alpha$

نهدف لاستبدال كل ال PR البينية لنستطيع الوصول ل α مباشرة

- نقول عن (A, B) زوج أحادي إذا كان أي A تؤدي إلى B بعد عدة خطوات
- كل زوج (A, A) هو زوج أحادي، وعلى نفس السياق إذا كان هناك قاعدة $A \rightarrow B$ إذا (A, B) زوج أحادي
- نتيجة: إذا كان كل من (A, B) و (B, C) زوج أحادي فإن (A, C) زوجاً أحادياً

الخوارزمية:

لدينا القاعدة $G=(V,T,P,S)$ ونهدف لبناء القاعدة $G1=(V,T,P1,S)$ خالية من القواعد الأحادية.

- نوجد كل الأزواج الأحادية في G
- لكل زوج أحادي نقوم ب:
- إضافة ل P1 قاعدة جديدة $A \rightarrow \alpha$ من أجل كل $B \rightarrow \alpha$ (ليست قاعدة أحادية)
- إذا كانت النتيجة موجودة في P1 مسبقاً لا نقوم بالتكرار

مثال:

لدينا ال PRs التالية:

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow M, M \rightarrow N, N \rightarrow a$$

نلاحظ الأزواج الأحادية هي التالية: $Y \rightarrow Z, Z \rightarrow M, M \rightarrow N$

و لدينا $N \rightarrow a$ بالتالي نبدأ بالتعويض في M فينتج لدينا قاعدة جديدة، تصبح ال PRs:

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$$

ثم نعوض M، فيصبح لدينا:

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$$

ثم نعوض Z، فيصبح لدينا:

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$$

هكذا قمنا بإزالة جميع الأزواج الأحادية

لكن نلاحظ أن Z و M و N لا يمكن الوصول إليهم من أي حالة أخرى لذلك بإمكاننا الاستغناء عنهم فتصبح ال PRs النهائية:

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b$$

unreachable states: هي الحالات التي لا يمكن الوصول إليها من الحالة الابتدائية

3. إزالة القواعد الفارغة Removal of Null Productions

المتغير nullable variable في ال CFG هو متغير non-terminal يمكن ان يؤدي للسلسلة الفارغة empty string إما بشكل مباشر $A \rightarrow \epsilon$ او بعد سلسلة من ال PR

الخطوات:

1. نحدد جميع ال nullable variables حيث $A \Rightarrow^* \epsilon$
 2. استبدال ال nullable variables في جميع المواضع التي تظهر فيها نقوم باستبدالهم في أي PR إذا كانوا في الجهة اليمينية إما ب ϵ أو نتركهم كما هم
 3. إضافة PRs جديدة وهي القواعد التي استنتجناها في الخطوة السابقة
 4. نزيل القواعد الفارغة
- نزيل كل القواعد من الشكل $A \rightarrow \epsilon$ إلا للحالة الابتدائية إذا كانت لازمة.

مثال:

لدينا ال PRs التالية :

$$S \rightarrow ABAC, A \rightarrow aA|\epsilon, B \rightarrow bB|\epsilon, C \rightarrow c$$

1- نحدد ال nullable variables:

- A is nullable (direct: $A \rightarrow \epsilon$).
- B is nullable (direct: $B \rightarrow \epsilon$).
- C is not nullable.

2- نحذف $A \rightarrow \epsilon$

أولا سنعوّض A في S بجميع الاحتمالات الممكنة، فنحصل على:

$$S \rightarrow ABAC \Rightarrow S \rightarrow ABC | BAC | BC$$

و هي مجموعة احتمالات تعويض A في مكان واحد او الاثنين معا

نقوم بتعديل ال PR الخاصة ب A : $A \rightarrow aA|a$

3- وبنفس الطريقة ل B لكن هنا نأخذ بعين الاعتبار ال PRs الجديدة الناتجة عن تعويض A أيضاً:

$$S \rightarrow ABAC \Rightarrow S \rightarrow ABC | BAC | BC$$

$$S \rightarrow AAC | AC | C$$



نقوم بتعديل ال PR الخاصة بـ B : $B \rightarrow bB \mid b$

تصبح ال PRs النهائية:

$$S \rightarrow ABAC \mid ABC \mid BAC \mid BC \mid AAC \mid AC \mid C$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

$$C \rightarrow c$$

القاعدة فعليا أصبحت أعقد بعد التبسيط و هذا يبين مدى أهمية ال ϵ و أن وجوده يكون أبسط

النهاية



لا تيأس إذا تعثرت، فكل العظماء
واجهوا الفشل قبل أن يحققوا النجاح